



TPI-PROGRAMACIÓN II

Sistema de Gestión de Pedidos y Comercio

Abril Duarte

Introducción

El presente trabajo práctico integrador consiste en el desarrollo de una aplicación orientada a la gestión integral de un sistema comercial, abarcando el control de clientes, productos, categorías y pedidos.

El objetivo principal es aplicar de manera práctica los conceptos fundamentales de la Programación Orientada a Objetos, garantizando un diseño modular, mantenible y con un control de errores adecuado frente a entradas inválidas.

Arquitectura y estructura del código

La solución fue organizada respetando la estructura modular en paquetes dentro de "integrado.prog2", separando las responsabilidades de forma clara:

- integrado.prog2: Contiene la clase ejecutable principal "Main.java" con el menú interactivo por consola y la interacción con el usuario.
- integrado.prog2.config: Contiene las clases de configuración y conexión del sistema:
 - ConfiguracionSistema.java (Formatos, moneda y encabezados).
 - ConexionDB.java (Gestión de la conexión al origen de datos).
- integrado.prog2.entities: Contiene las clases de dominio con sus atributos privados, getters y setters:
 - Base.java (Clase abstracta de la cual heredan todas las entidades).
 - Categoria.java
 - Producto.java
 - Usuario.java
 - Pedido.java
 - DetallePedido.java
- integrado.prog2.enums: Tipos enumerados para mantener el tipado fuerte en el sistema:
 - Estado.java (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO).
 - FormaPago.java (TARJETA, TRANSFERENCIA, EFECTIVO).

- Rol.java (ADMIN, USUARIO).
- integrado.prog2.exceptions: Excepciones personalizadas creadas para la gestión de errores del dominio:
 - ValidacionNegocioException.java
 - EntidadNoEncontradaException.java

Pilares de la programación orientada a objetos y sus reglas de negocio

- Encapsulamiento y Herencia: Todos los atributos de los modelos se declararon como privados (``private``). Las entidades heredan de la clase padre Base para compartir atributos comunes como el ``id`` y el estado de baja lógica "eliminado".
- Baja Lógica: Para preservar la integridad referencial y el historial de transacciones, no se realizan borrados físicos. Al eliminar una entidad, su atributo ``eliminado`` cambia a ``true``, quedando excluida de los listados activos y operaciones posteriores.
- Gestión de Excepciones y Reglas de Negocio:
 - Control de unicidad para evitar duplicados en nombres de categorías y mail de usuarios.
 - Verificación de stock disponible al agregar productos a un pedido, aplicando el descuento automático de unidades.
 - Lanzamiento de ``ValidacionNegocioException`` ante montos o stock negativos.
 - Bloques ``try-catch`` para evitar la interrupción del programa ante ingresos alfabéticos en campos numéricos.

Casos de pruebas y demostraciones en video

Durante la demostración práctica grabada en video se verifican las siguientes operaciones:

- Gestión de Categorías, Productos y Usuarios (Altas, bajas lógicas y validación de duplicados).
- Alta de Pedidos vinculando usuario, forma de pago, producto y cantidad, con cálculo automático del total y actualización de stock.
- Actualización de estado del pedido y aplicación de baja lógica.

- Pruebas de borde: Ingreso de datos alfabéticos en campos de precio/stock e intento de pedido con stock insuficiente, demostrando la captura de excepciones sin caída del sistema.

Enlace al video demostrativo

Youtube:

<https://www.youtube.com/watch?v=oHU736O25uM>